

MODUL II

ORM Eloquent dan CRUD

Tujuan

- ◆ Mahasiswa dapat membuat dan menjalankan migration.
- ◆ Mahasiswa dapat membuat Model Eloquent dengan atribut `$fillable` dan mendefinisikan relasi `hasMany` / `belongsTo`.
- ◆ Mahasiswa dapat menghasilkan Resource Controller via Artisan dan mengisi setiap method untuk operasi CRUD.
- ◆ Mahasiswa dapat mendefinisikan route API (`Route::apiResource`) dan menghubungkannya dengan controller.
- ◆ Mahasiswa dapat menguji endpoint CRUD menggunakan Postman dan memverifikasi response JSON.

Materi

1. Konfigurasi `.env` & database.
 - 1.1. Buka file `.env` di root project. Ubah bagian database:
File: `inventory/.env`
`DB_CONNECTION=mysql`
`DB_HOST=127.0.0.1`
`DB_PORT=3306`
`DB_DATABASE=inventory_db`
`DB_USERNAME=root`
`DB_PASSWORD=`
 - 1.2. Pada Laragon → Menu MySQL → Admin (phpMyAdmin)
Buat database baru: `inventory_db`.
 - 1.3. Uji koneksi:
`php artisan migrate:status`
Jika gagal, cek kembali konfigurasi `.env` dan ulangi perintah ini.
2. ORM & Eloquent
 - 2.1. ORM (Object-Relational Mapping) memetakan tabel database ke class Model PHP.
 - 2.2. Laravel menggunakan Eloquent: setiap table diwakili dengan satu Model.
 - 2.3. Manfaat: operasi CRUD dilakukan lewat method PHP dan sudah tidak lagi menggunakan SQL murni.
3. Migration
 - 3.1. Migration file terletak di `database/migrations/`.
 - 3.2. Format nama: `timestamp_create_xxx_table.php`.

3.3. Method `up()` untuk membuat/ubah tabel, `down()` untuk rollback.

3.4. Artisan commands:

- Buat migration baru:
`php artisan make:migration create_items_table --create=items`
- Jalankan semua migration:
`php artisan migrate`
- Jika migration ingin dikembalikan atau rollback:
`php artisan migrate:rollback`

4. Model

4.1. Direktori: `app/Models/`, contoh `app/Models/Item.php`.

4.2. Artisan commands:

- Buat model:
`php artisan make:model Item`
- Buat migration sekaligus dengan model:
`php artisan make:model Item -m`

4.3. Property penting di Model:

- `protected $fillable = ['field1', 'field2', ...];`
- `protected $guarded = [];`
- `public $timestamps = true;`

5. Relasi antar model

5.1. One-to-Many (1-N)

- di `Category.php`:

```
public function items() {  
    return $this->hasMany(Item::class);  
}
```
- di `Item.php`:

```
public function category() {  
    return $this->belongsTo(Category::class);  
}
```

5.2. Many-to-Many (N-M, pivot)

contoh: relasi data products dan data tags melalui pivot table dengan `product_tag`, Pada model menggunakan metode `belongsToMany()`.

6. CRUD Controller

6.1. Manual routes (contoh di `routes/api.php`):

```
Route::get('items', 'ItemController::class@index');  
Route::post('items', 'ItemController::class@store');  
...
```

6.2. Resource controller:

```
Route::apiResource('items', 'ItemController::class');
```

otomatis menyediakan index, show, store, update, destroy.

6.3. Artisan:

```
php artisan make:controller ItemController --api --  
model=Item  
perintah ini menghasilkan potongan kode method di  
app/Http/Controllers/ItemController.php.
```

Praktikum

1. Pastikan Anda switch dari branch main ke branch feature/models:

```
cd C:\laragon\www\inventory  
git checkout main  
git pull origin main  
git checkout -b feature/models
```

2. Buat Migration & Model

2.1. Migration tabel categories:

```
php artisan make:migration create_categories_table --  
create=categories  
  
File: 2026_05_03_125045_create_categories_table.php  
<?php  
...  
return new class extends Migration {  
    public function up() {  
        Schema::create('categories', function (Blueprint $table)  
        {  
            $table->id();  
            $table->string('name')->unique();  
            $table->timestamps();  
        });  
    }  
    public function down() {  
        Schema::dropIfExists('categories');  
    }  
};
```

2.2. Migration tabel items:

```
php artisan make:migration create_items_table --  
create=items  
  
File: 2026_05_03_125045_create_items_table.php  
<?php  
...  
return new class extends Migration {  
    public function up() {  
        Schema::create('items', function (Blueprint $table) {  
            $table->id();  
            $table->string('name');  
            $table->unsignedInteger('quantity')->default(0);  
            $table->decimal('price', 12, 2)->default(0);  
            $table->foreignId('category_id')  
                ->constrained()
```

```

        ->onDelete('cascade');
        $table->timestamps();
    });
}
public function down() {
    Schema::dropIfExists('items');
}
};

```

2.3. Jalankan migration:

```
php artisan migrate
```

3. Buat Model & definisikan fillable + relasi

3.1. Category model:

```
php artisan make:model Category
```

File: app/Models/Category.php

```

<?php
...
class Category extends Model {
    use HasFactory;
    protected $fillable = ['name'];
    public function items() {
        return $this->hasMany(Item::class);
    }
}

```

3.2. Item model:

```
php artisan make:model Item
```

File: app/Models/Item.php

```

<?php
...
class Item extends Model {
    use HasFactory;
    protected $fillable = [
        'name',
        'quantity',
        'price',
        'category_id',
    ];
    public function category() {
        return $this->belongsTo(Category::class);
    }
}

```

4. Buat Resource Controller untuk Item & Category

4.1. CategoryController:

```

php artisan make:controller CategoryController \
--api --model=Category

```

```

File: app/Http/Controllers/CategoryController.php
<?php
...
class CategoryController extends Controller {
    public function index() {
        return response()->json(Category::all());
    }
    public function store(Request $request) {
        $category = Category::create($request->all());
        return response()->json($category, 201);
    }
    public function show($id) {
        $category = Category::findOrFail($id);
        return response()->json($category);
    }
    public function update(Request $request, $id) {
        $category = Category::findOrFail($id);
        $category->update($request->all());
        return response()->json($category);
    }
    public function destroy($id) {
        Category::destroy($id);
        return response()->json(null, 204);
    }
}

```

4.2. ItemController:

```

php artisan make:controller ItemController \
--api --model=Item

```

```

File: app/Http/Controllers/ItemController.php
<?php
...
class ItemController extends Controller {
    public function index() {
        return response()->json(Item::with('category')->get());
    }
    public function store(Request $request) {
        $item = Item::create($request->all());
        return response()->json($item, 201);
    }
    public function show($id) {
        $item = Item::with('category')->findOrFail($id);
        return response()->json($item);
    }

    public function update(Request $request, $id) {
        $item = Item::findOrFail($id);
        $item->update($request->all());
        return response()->json($item);
    }
    public function destroy($id) {
        Item::destroy($id);
        return response()->json(null, 204);
    }
}

```

```
}  
}
```

5. Update routing

File: routes/api.php

<?php

...

Route::apiResource('categories', CategoryController::class);

Route::apiResource('items', ItemController::class);

6. Uji Endpoint di Postman

- Tarik semua data kategori

GET <http://127.0.0.1:8000/api/categories>

Pertemuan 2 / Category / All

GET <http://127.0.0.1:8000/api/categories> Send

Docs Params Authorization Headers (6) Body Scripts Settings Cookies

Query Params

| Key | Value | Description |
|-----|-------|-------------|
| Key | Value | Description |

Body Cookies Headers (7) Test Results 200 OK • 605 ms • 338 B Save Response

JSON Preview Visualize

```
1 [
2   {
3     "id": 1,
4     "name": "Electronics",
5     "created_at": "2026-05-03T23:47:34.000000Z",
6     "updated_at": "2026-05-03T23:47:34.000000Z"
7   }
8 ]
```

- Tarik satu data kategori berdasarkan id

GET <http://127.0.0.1:8000/api/categories/1>

Pertemuan 2 / Category / By Id

GET <http://127.0.0.1:8000/api/categories/1> Send

Docs Params Authorization Headers (6) Body Scripts Settings Cookies

Path Variables

| Key | Value | Description |
|-----|-------|-------------|
| id | 1 | Description |

Body Cookies Headers (7) Test Results 200 OK • 1.43 s • 336 B Save Response

JSON Preview Visualize

```
1 {
2   "id": 1,
3   "name": "Electronics",
4   "created_at": "2026-05-03T23:47:34.000000Z",
5   "updated_at": "2026-05-03T23:47:34.000000Z"
6 }
```

- Tambah data kategori
POST <http://127.0.0.1:8000/api/categories>

The screenshot shows a Postman interface for a POST request to `http://127.0.0.1:8000/api/categories`. The request body is in `x-www-form-urlencoded` format with a single key-value pair: `name: Electronics`. The response is a `201 Created` status with a response time of 504 ms and a body size of 341 B. The response body is shown in JSON format:

```
{
  "name": "Electronics",
  "updated_at": "2026-05-03T23:47:34.000000Z",
  "created_at": "2026-05-03T23:47:34.000000Z",
  "id": 1
}
```

- Ubah data kategori
PUT <http://127.0.0.1:8000/api/categories/1>

The screenshot shows a Postman interface for a PUT request to `http://127.0.0.1:8000/api/categories/:id`. The request body is in `x-www-form-urlencoded` format with a single key-value pair: `name: Electronics Edit`. The response is a `200 OK` status with a response time of 1.09 s and a body size of 341 B. The response body is shown in JSON format:

```
{
  "id": 1,
  "name": "Electronics Edit",
  "created_at": "2026-05-03T23:47:34.000000Z",
  "updated_at": "2026-05-03T23:58:45.000000Z"
}
```

- Hapus data kategori
DELETE <http://127.0.0.1:8000/api/categories/1>

The screenshot shows a Postman interface for a DELETE request to `http://127.0.0.1:8000/api/categories/:id`. The path variable `id` is set to `1`. The response is a `204 No Content` status with a response time of 1.11 s and a body size of 197 B. The response body is shown in raw format as:

```
1
```

- Tarik semua data item
GET <http://127.0.0.1:8000/api/items>

Pertemuan 2 / Item / All

GET <http://127.0.0.1:8000/api/items> Send

Docs Params Authorization Headers (6) Body Scripts Settings Cookies

Query Params

| Key | Value | Description |
|-----|-------|-------------|
| Key | Value | Description |

Body Cookies Headers (7) Test Results 200 OK • 1.05 s • 508 B Save Response

JSON Preview Visualize

```
1 [
2   {
3     "id": 1,
4     "name": "Laptop",
5     "quantity": 10,
6     "price": "1500.00",
7     "category_id": 1,
8     "created_at": "2026-05-04T00:14:46.000000Z",
9     "updated_at": "2026-05-04T00:14:46.000000Z",
10    "category": {
11      "id": 1,
12      "name": "Electronics",
13      "created_at": "2026-05-04T00:14:38.000000Z",
14      "updated_at": "2026-05-04T00:14:38.000000Z"
15    }
16  }
17 ]
```

- Tarik satu data item berdasarkan id
GET <http://127.0.0.1:8000/api/items/1>

Pertemuan 2 / Item / By Id

GET <http://127.0.0.1:8000/api/items/1> Send

Docs Params Authorization Headers (6) Body Scripts Settings Cookies

Path Variables

| Key | Value | Description |
|-----|-------|-------------|
| id | 1 | Description |

Body Cookies Headers (7) Test Results 200 OK • 910 ms • 506 B Save Response

JSON Preview Visualize

```
1 {
2   "id": 1,
3   "name": "Laptop",
4   "quantity": 10,
5   "price": "1500.00",
6   "category_id": 1,
7   "created_at": "2026-05-04T00:14:46.000000Z",
8   "updated_at": "2026-05-04T00:14:46.000000Z",
9   "category": {
10     "id": 1,
11     "name": "Electronics",
12     "created_at": "2026-05-04T00:14:38.000000Z",
13     "updated_at": "2026-05-04T00:14:38.000000Z"
14   }
15 }
```

- Tambah data item
POST <http://127.0.0.1:8000/api/items>

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** <http://127.0.0.1:8000/api/items>
- Body Type:** x-www-form-urlencoded
- Body Data:**

| Key | Value | Description |
|-------------|--------|-------------|
| name | Laptop | |
| quantity | 10 | |
| price | 1500 | |
| category_id | 1 | |
- Response:** 201 Created, 1.25 s, 385 B
- Response Body (JSON):**

```
{  "id": 1,  "name": "Laptop",  "quantity": "10",  "price": "1500",  "category_id": "1",  "updated_at": "2026-05-04T00:14:46.000000Z",  "created_at": "2026-05-04T00:14:46.000000Z"}
```

- Ubah data item
PUT <http://127.0.0.1:8000/api/items/1>

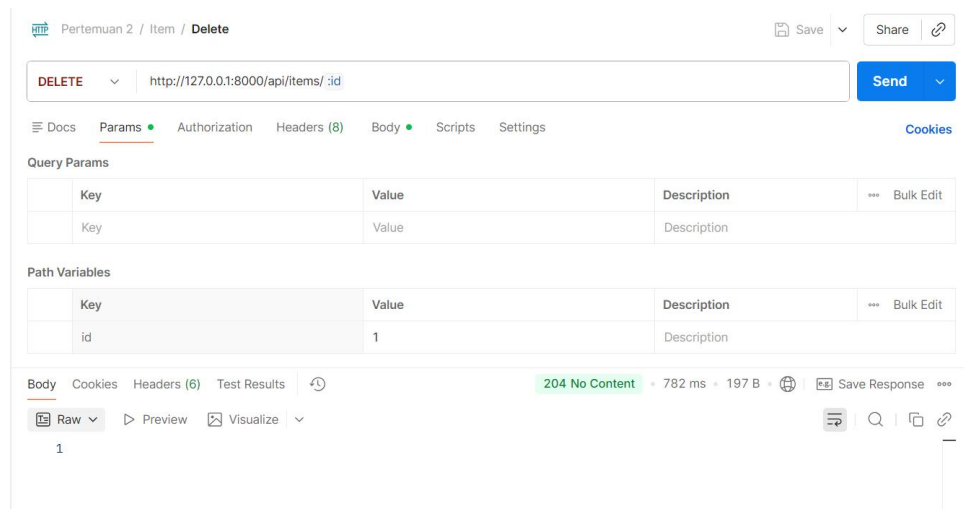
The screenshot shows a REST client interface with the following details:

- Method:** PATCH
- URL:** <http://127.0.0.1:8000/api/items/1>
- Body Type:** x-www-form-urlencoded
- Body Data:**

| Key | Value | Description |
|-------------|-------------|-------------|
| name | Laptop Edit | |
| quantity | 20 | |
| price | 15000 | |
| category_id | 1 | |
- Response:** 200 OK, 1.23 s, 386 B
- Response Body (JSON):**

```
{  "id": 1,  "name": "Laptop Edit",  "quantity": "20",  "price": "15000",  "category_id": "1",  "updated_at": "2026-05-04T00:17:24.000000Z",  "created_at": "2026-05-04T00:14:46.000000Z"}
```

- Hapus data item
DELETE <http://127.0.0.1:8000/api/items/1>



7. Commit hasil

```
git add .
git commit -m "Add migrations, models, relations, resource controllers for items & categories"
git push -u origin feature/models
```

8. Pull Request

8.1. Buka Repository di GitHub

- Pergi ke <https://github.com/USERNAME/inventory>
- Kamu akan melihat notifikasi kuning: "**feature/modes had recent pushes**" → klik "**Compare & pull request**"

8.2. Isi Form Pull Request

- Base branch: main (branch tujuan merge)
- Compare branch: feature/models (branch kamu)
- Title: Isi judul PR
- Setelah form diisi, klik tombol "Create pull request"

8.3. Review Kode (Melihat Perubahan)

- Klik tab "Files changed"
- Lihat perubahan kode:
 - Warna hijau = baris kode yang ditambah
 - Warna merah = baris kode yang dihapus

8.4. Merge Pull Request

- Klik tombol "Merge pull request"
- Klik "Confirm merge"